MixBytes()

MEV-X Homelander Security Audit Report

MAY 12, 2026

Table of Contents

1. Introduction	**3**
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	4
1.4 Security Assessment Methodology	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	**12**
2.1 Critical	12
2.2 High	12
2.3 Medium	12
M-1 Executor-initiated swaps are not excluded from afterSwap recursion	12
M-2 Users can strategically underprovision gas to avoid paying for arbitrage execution	13
M-3 External operations have no gas limits and can revert user swaps	14
2.4 Low	15
L-1 Local dynamicFee config can silently diverge from the pool's real fee mode	15
L-2 Test-only hook validation bypass left in production code	16
L-3 Malformed successful router response can still revert the user swap	17
L-4 Single-step ownership transfer and renouncement create unnecessary governance risk	18
L-5 Missing zero-address checks can disable critical configuration paths	19
L-6 Silent failure paths have no native observability	20
L-7 dynamicFee_ validation only covers the flag-set branch	21
L-8 distributeProfit revert in nested empty catch can strand profit at the distributor	22

L-9 profitDistributor is re-read after the executor call instead of being cached once 23

L-10 setMevxRouter() can silently abandon pool registrations created by earlier _afterInitialize() calls 24

L-11 setConfigId() and setProfitDistributor() should be updated atomically 25

3. About MixBytes **26**

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

MEV-X Homelander is an on-chain MEV internalization module for DEX integrations that uses a Uniswap v4 hook to inspect swaps after execution and trigger an external arbitrage stack when profitable opportunities arise. The audited repository contains the Uniswap v4 plugin and its integration interfaces, while route construction, route execution, and profit allocation are delegated to external contracts. Within this design, the plugin acts primarily as an integration and control layer that registers pools, applies optional dynamic-fee behavior, and forwards post-swap context to the MEV-X execution stack.

Our review covered both the hook's core control flow and its interactions with the external MEV-X Router, MEV-X Executor, and Profit Distributor components. We also assessed the code against a standard checklist covering access control, failure handling, trust boundaries, callback correctness, and common integration-level issues specific to Uniswap v4 hooks. The project-specific attack vectors considered during the review included the following:

- **Failure propagation across external dependencies.** Particular attention was given to whether failures in the MEV-X Router, MEV-X Executor, or Profit Distributor could propagate through the hook path and cause user-facing pool operations to revert.
- **Configuration-induced denial of service.** The review considered whether incorrect operational or deployment configuration could place the integration into a state in which normal user interactions revert or degrade unexpectedly.
- **User-driven disruption of arbitrage execution.** The review also considered whether a user could interfere with the arbitrage module by shaping swap parameters or otherwise influencing transaction context in a way that suppresses, breaks, or biases the intended execution path.

The external MEV-X stack contracts, namely the MEV-X Router, MEV-X Executor, and Profit Distributor, as well as the surrounding Uniswap v4 hook execution environment, were out of scope of this audit; they were only reviewed through their interfaces, integration assumptions, and available documentation.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **Clear integration boundary.** The code keeps the audited component narrowly focused on pool registration, callback handling, and forwarding execution context to external systems. This reduces the amount of protocol logic held inside the hook itself, but it also means the security posture of the integration depends heavily on correct assumptions about trusted downstream components.

- **Fail-open intent with partial enforcement.** The plugin is structured so that failures in pool registration, route construction, execution, and profit distribution are generally swallowed rather than bubbling up into the user swap path. That design is directionally sound for preserving swap liveness, but it should be reviewed carefully because malformed downstream behavior and gas-related edge cases can still undermine the intended fail-open property.
- **Observability and documentation should be strengthened.** Several important failure paths intentionally do not revert, which makes operational degradation harder to detect on-chain without dedicated telemetry. The project would benefit from clearer documentation of trust assumptions, recipient semantics, and deployment requirements, alongside additional monitoring signals for suppressed failures.

1.3 Project Overview

Summary

Title	Description
Client	MEV-X
Category	DEX
Project	Homelander
Type	Solidity
Platform	EVM
Timeline	20.04.2026 - 12.05.2026

Scope of Audit

File	Link
contracts/Constants.sol	contracts/Constants.sol
contracts/HomelanderUniV4Plugin.sol	contracts/HomelanderUniV4Plugin.sol

Versions Log

Date	Commit Hash	Note
20.04.2026	3871d01a62a64aaf5ece68be29bde4044b56abdd	Initial Commit
23.04.2026	c73c2342a87c1b53e97c902d18c39610a5ddc2cf	Commit for the re-audit
27.04.2026	f725ae0dcf3191921dafe95d416ead4a666e3bb0	Commit for the 2nd re-audit
27.04.2026	a908427a1d0ece97b5159397d22eb32be615e936	Commit for the 3rd re-audit
28.04.2026	a89d5f311c58c000549566f742ec72081531ba84	Commit for the 4th re-audit
12.05.2026	136462c3e651ef010fce6c10e22bd7c3744843d5	Commit for the 5th re-audit

Mainnet Deployments

File	Address	Blockchain
HomelanderUniV4Plugin.sol	0xc62417134aea63F0aF800B83A62a06e6bD7690c0	Ethereum
HomelanderUniV4Plugin.sol	0xBFa8b722aaB2b2cB989B330Bc6e5747014F890c0	Arbitrum
HomelanderUniV4Plugin.sol	0xB0334F544160d73469E4C1DB9693d2c98C0a90c0	Base
HomelanderUniV4Plugin.sol	0xE2e975Bc2dA43EAe5bA46AcD7e779C34822e50c0	Polygon
HomelanderUniV4Plugin.sol	0x270C96A3f6C461EE9C882b7aE5D7D3dFA37690C0	BSC
HomelanderUniV4Plugin.sol	0x918e7492Fa2A5EFD5326d555E484381a5765D0c0	Monad

1.4 Security Assessment Methodology

Project Flow

Stage	Scope of Work
Interim audit	**Project Architecture Review:** · Review project documentation · Conduct a general code review · Perform reverse engineering to analyze the project's architecture based solely on the source code · Develop an independent perspective on the project's architecture · Identify any logical flaws in the design OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROJECT AND IDENTIFY POTENTIAL SECURITY RISKS.
	Code Review with a Hacker Mindset: · Each team member independently conducts a manual code review, focusing on identifying unique vulnerabilities. · Perform collaborative audits (pair auditing) of the most complex code sections, supervised by the Team Lead. · Develop Proof-of-Concepts (PoCs) and conduct fuzzing tests using tools like Foundry, Hardhat, and BOA to uncover intricate logical flaws. · Review test cases and in-code comments to identify potential weaknesses. OBJECTIVE: IDENTIFY AND ELIMINATE THE MAJORITY OF VULNERABILITIES, INCLUDING THOSE UNIQUE TO THE INDUSTRY.
	Code Review with a Nerd Mindset: · Conduct a manual code review using an internally maintained checklist, regularly updated with insights from past hacks, research, and client audits. · Utilize static analysis tools (e.g., Slither, Mythril) and vulnerability databases (e.g., Solodit) to uncover potential undetected attack vectors. OBJECTIVE: ENSURE COMPREHENSIVE COVERAGE OF ALL KNOWN ATTACK VECTORS DURING THE REVIEW PROCESS.

Stage	Scope of Work
	Consolidation of Auditors' Reports: · Cross-check findings among auditors · Discuss identified issues · Issue an interim audit report for client review OBJECTIVE: COMBINE INTERIM REPORTS FROM ALL AUDITORS INTO A SINGLE COMPREHENSIVE DOCUMENT.
Re-audit	**Bug Fixing & Re-Audit:** · The client addresses the identified issues and provides feedback · Auditors verify the fixes and update their statuses with supporting evidence · A re-audit report is generated and shared with the client OBJECTIVE: VALIDATE THE FIXES AND REASSESS THE CODE TO ENSURE ALL VULNERABILITIES ARE RESOLVED AND NO NEW VULNERABILITIES ARE ADDED.
Final audit	**Final Code Verification & Public Audit Report:** · Verify the final code version against recommendations and their statuses · Check deployed contracts for correct initialization parameters · Confirm that the deployed code matches the audited version · Issue a public audit report, published on our official GitHub repository · Announce the successful audit on our official X account OBJECTIVE: PERFORM A FINAL REVIEW AND ISSUE A PUBLIC REPORT DOCUMENTING THE AUDIT.

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likehood: High	Critical	High	Medium
Likehood: Medium	High	Medium	Low
Likehood: Low	Medium	Low	Low

Impact

- **High** – Theft from 0.5% OR partial/full blocking of funds (>0.5%) on the contract without the possibility of withdrawal OR loss of user funds (>1%) who interacted with the protocol.
- **Medium** – Contract lock that can only be fixed through a contract upgrade OR one-time theft of rewards or an amount up to 0.5% of the protocol's TVL OR funds lock with the possibility of withdrawal by an admin.
- **Low** – One-time contract lock that can be fixed by the administrator without a contract upgrade.

Likelihood

- **High** – The event has a 50–60% probability of occurring within a year and can be triggered by any actor (e.g., due to a likely market condition that the actor cannot influence).
- **Medium** – An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed to minimize potential risks.
- **Medium** – Recommended to be fixed to enhance security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and effectiveness.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and no longer impact its security.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but it has not been fully resolved.
- **Acknowledged** – The recommended fixes have not yet been implemented, and the finding remains unresolved or does not require code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	3
Low	11

Findings Statuses

ID	Finding	Severity	Status
M-1	Executor-initiated swaps are not excluded from afterSwap recursion	Medium	Fixed
M-2	Users can strategically underprovision gas to avoid paying for arbitrage execution	Medium	Fixed
M-3	External operations have no gas limits and can revert user swaps	Medium	Fixed
L-1	Local dynamicFee config can silently diverge from the pool's real fee mode	Low	Acknowledged
L-2	Test-only hook validation bypass left in production code	Low	Fixed
L-3	Malformed successful router response can still revert the user swap	Low	Fixed
L-4	Single-step ownership transfer and renouncement create unnecessary governance risk	Low	Fixed
L-5	Missing zero-address checks can disable critical configuration paths	Low	Fixed
L-6	Silent failure paths have no native observability	Low	Acknowledged

L-7	dynamicFee_ validation only covers the flag-set branch	Low	Fixed
L-8	distributeProfit revert in nested empty catch can strand profit at the distributor	Low	Acknowledged
L-9	profitDistributor is re-read after the executor call instead of being cached once	Low	Fixed
L-10	setMevxRouter() can silently abandon pool registrations created by earlier _afterInitialize() calls	Low	Acknowledged
L-11	setConfigId() and setProfitDistributor() should be updated atomically	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

M-1	Executor-initiated swaps are not excluded from afterSwap recursion		
Severity	Medium	**Status**	Fixed in c73c2342

Description
The hook does not exclude executor-initiated swaps from _afterSwap(). If route execution triggers another swap through a pool using the same hook, the nested swap may enter arbitrage discovery and execution again. This can lead to unexpected nested arbitrage behavior, make execution flow harder to reason about, and push the transaction toward gas-limit failures or reverts caused by nonReentrant-style guards in downstream components.

Recommendation
Consider excluding executor-initiated swaps from afterSwap processing at the code level. If nested execution is not an intended feature, the hook should explicitly short-circuit this path to avoid unexpected behavior, downstream guard-triggered reverts, and unnecessary gas consumption.

Client's Commentary:
Fixed c73c2342a87c1b53e97c902d18c39610a5ddc2cf. In _afterSwap, when sender == address(mevxExecutor) the hook short-circuits with return (BaseHook.afterSwap.selector, 0) before any route construction or execution.

M-2	Users can strategically underprovision gas to avoid paying for arbitrage execution		
Severity	Medium	Status	Fixed in c73c2342

Description

Because the arbitrage path runs inside the user's swap transaction, its gas cost is also borne by the user. This creates an incentive for users to submit swaps with a deliberately tight gas limit so the core swap still completes while the post-swap arbitrage path fails or is skipped. In practice, automated gas-limit search and estimation algorithms are also likely to converge toward this lower-gas execution path, systematically biasing execution against arbitrage.

Recommendation

If arbitrage execution is a required part of the design, consider enforcing a dedicated gas reserve or otherwise making the arbitrage path independent from user-supplied gas budgeting. At minimum, this behavior should be treated as an expected integration constraint rather than assuming users will voluntarily fund the arbitrage branch.

Client's Commentary:

Fixed c73c2342a87c1b53e97c902d18c39610a5ddc2cf. We introduced a configurable minGasLeft parameter (defaults to 0, capped at 1,500,000). In _afterSwap, the hook calls require(gasleft() >= minGasLeft, …) after the initial arb check, so if the caller underprovisions gas the entire swap reverts. This removes the incentive to submit swaps with a tight gas limit — users must supply enough gas to cover the arbitrage path, or their swap fails.

M-3	External operations have no gas limits and can revert user swaps		
Severity	Medium	**Status**	Fixed in a908427a

Description

The contract does not apply explicit gas limits to its external operational calls. As a result, a buggy or gas-heavy router, executor, or distributor can consume enough gas inside the hook path to make the parent frame run out of gas as well. In that case, existing `try/catch` wrappers do not reliably protect the surrounding swap flow, and the user swap itself can still revert.

Recommendation

Consider applying explicit gas budgets to external calls on the critical hook path, and treat gas exhaustion of downstream components as a user-swap liveness risk rather than assuming `try/catch` is sufficient protection.

2.4 Low

L-1	Local dynamicFee config can silently diverge from the pool's real fee mode		
Severity	Low	Status	Acknowledged

Description

The hook enables dynamic-fee behavior based on its local `dynamicFee` configuration, but Uniswap v4 only applies fee overrides for pools that are actually configured as dynamic-fee pools. As a result, the contract can attempt to use dynamic-fee logic against a pool that does not support it, creating a silent mismatch between expected and actual behavior.

Recommendation

Validate the pool fee configuration on the integration path and only enable dynamic-fee functionality for pools that actually support it. If the target pool is static-fee, the contract should disable dynamic-fee-specific logic instead of attempting to apply fee overrides.

Client's Commentary:

Acknowledged

L-2	Test-only hook validation bypass left in production code		
Severity	Low	**Status**	Fixed in c73c2342

Description

The contract overrides `BaseHook.validateHookAddress()` with an empty implementation, disabling Uniswap v4's built-in check that the deployed hook address matches the permission bits declared in `getHookPermissions()`. While this may be convenient in tests, keeping this bypass in the production contract removes an important deployment-time invariant and can lead to silent misbehavior if the hook is deployed to an address with incorrect bits.

Recommendation

Remove the validation bypass before production deployment. If this behavior is needed for tests, implement it in a dedicated derived test contract instead of the production contract.

Client's Commentary:

Fixed c73c2342a87c1b53e97c902d18c39610a5ddc2cf

L-3	Malformed successful router response can still revert the user swap		
Severity	Low	**Status**	Fixed in c73c2342

Description

The contract uses a low-level `call` to `mevxRouter` so router reverts do not bubble up and break the user swap. However, if the call returns `success == true` with malformed ABI data, the subsequent `abi.decode(...)` still reverts inside the hook. This breaks the intended fail-open behavior and allows router to revert user swaps without explicitly reverting itself.

```
(bool success, bytes memory returnData) = address(mevxRouter).call(callData);

if (success && returnData.length > 0) {
    (isArbPossible, profitToken, pools, amountIn, encodedRoute) =
        abi.decode(returnData, (bool, address, address[], uint256, bytes));
}
```

Recommendation

Treat malformed successful responses as a non-fatal condition as well. Validate the returned data before decoding.

Client's Commentary:

We've additionally tightened the check by comparing the returned data length against the minimum possible length of the expected structure. We deliberately avoided wrapping the decode in a try/catch on an external call to avoid the redundant gas usage.

L-4	Single-step ownership transfer and renouncement create unnecessary governance risk		
Severity	Low	**Status**	Fixed in c73c2342

Description

The contract relies on the owner to maintain and recover critical external wiring, but still uses single-step ownership transfer and leaves `renounceOwnership()` enabled. This creates two avoidable governance risks: ownership can be transferred to a wrong or unreachable address with no acceptance step, and ownership can be permanently renounced even though the system still depends on admin-controlled recovery paths for router, executor, and distributor changes.

Recommendation

Use `Ownable2Step` for ownership transfers and override `renounceOwnership()` to revert. If the plugin requires ongoing owner-managed dependency rotation, ownership should not be permanently droppable.

Client's Commentary:

Fixed c73c2342a87c1b53e97c902d18c39610a5ddc2cf

L-5	Missing zero-address checks can disable critical configuration paths		
Severity	Low	**Status**	Fixed in c73c2342

Description

The constructor and admin setters do not consistently reject zero addresses for critical roles and external dependencies. This means the contract can be deployed with, or later switched to, invalid wiring such as a zero owner, router, executor, or profit distributor, resulting in permanent loss of admin control or silent degradation of core functionality.

Recommendation

Add explicit zero-address validation for all critical constructor parameters and admin-settable dependency addresses. If disabling a dependency is ever intended, it should be handled through a dedicated and explicit control path rather than by assigning `address(0)`.

Client's Commentary:

Fixed c73c2342a87c1b53e97c902d18c39610a5ddc2cf

L-6	Silent failure paths have no native observability		
Severity	Low	**Status**	Acknowledged

Description

The contract intentionally swallows several operational failure paths, but does not emit any native events when those paths are taken. As a result, failed pool registration, failed route execution, or failed profit distribution can be indistinguishable from normal no-op behavior unless operators rely on external tracing or indirect signals.

Recommendation

Emit dedicated operational events for swallowed failure paths so monitoring systems can distinguish expected no-op execution from degraded behavior.

Client's Commentary:

Dedicated operational events are emitted inside MevxRouter, MevxExecutor contracts

L-7	dynamicFee_ validation only covers the flag-set branch		
Severity	Low	**Status**	Fixed in c73c2342

Description

The constructor validates dynamicFee_ only when the dynamic-fee flag is set. If the flag is accidentally omitted, non-zero values are still silently accepted and the contract is deployed with dynamic-fee behavior effectively disabled. This can leave the plugin running with static-fee behavior even though the operator intended to configure dynamic-fee logic.

Recommendation

Validate dynamicFee_ on both branches. If the dynamic-fee flag is unset, the constructor should reject non-zero encodings instead of silently accepting them.

Client's Commentary:

Fixed c73c2342a87c1b53e97c902d18c39610a5ddc2cf

L-8	distributeProfit revert in nested empty catch can strand profit at the distributor		
Severity	Low	**Status**	Acknowledged

Description

The post-swap flow is not atomic: if executeRoute(...) succeeds and transfers profit to the distributor, but distributeProfit(...) then reverts, the nested empty catch silently swallows the failure. As a result, profit can accumulate at the distributor without being settled under the intended configId, while the user swap still completes successfully and the plugin records no native signal of the failure.

Recommendation

Emit a dedicated failure event from the distributeProfit(...) catch path and consider adding an explicit recovery or retry mechanism for unsettled profit so failed distribution does not silently drift into an untracked distributor balance.

Client's Commentary:

Acknowledged

L-9	profitDistributor is re-read after the executor call instead of being cached once		
Severity	Low	**Status**	Fixed in c73c2342

Description

_afterSwap() reads profitDistributor more than once around an external executor call instead of caching it to a local variable. Besides creating unnecessary storage reads on a hot path, this also makes the flow less robust because the address passed to the executor and the address used for subsequent distribution are not explicitly pinned to the same in-memory value for the duration of the function.

Recommendation

Cache profitDistributor once at the start of _afterSwap() and reuse the local variable for both the executor recipient and the distribution call. This reduces gas usage and makes the flow more explicit and consistent.

Client's Commentary:

Fixed c73c2342a87c1b53e97c902d18c39610a5ddc2cf

L-10	setMevxRouter() can silently abandon pool registrations created by earlier _afterInitialize() calls		
Severity	Low	Status	Acknowledged

Description

The plugin registers pools into the current router during _afterInitialize(), but setMevxRouter() can later replace that router without replaying or migrating those registrations. Since the plugin does not keep an on-chain registry of previously initialized pools, the new router may start operating without awareness of already existing pools, causing silent loss of functionality or broken behavior after migration.

Recommendation

Treat router replacement as a migration procedure rather than a simple pointer update. This can be addressed by keeping a replayable on-chain list of registered pools, requiring an explicit migration step, or otherwise blocking router rotation until prior pool registrations have been re-established.

Client's Commentary:

If pools is not registered it will be registered at first interaction with MevxRouter

L-11	setConfigId() and setProfitDistributor() should be updated atomically		
Severity	Low	**Status**	Acknowledged

Description

The plugin updates configId and profitDistributor through two independent admin setters even though these values are logically coupled on the integration path. A configuration identifier is meaningful only relative to the active distributor implementation and its internal policy table, so applying the two changes in separate transactions creates an intermediate state in which one value may already be updated while the other still points to the previous configuration domain. In that window, profit distribution can fail or be routed under unintended assumptions without any atomicity guarantee at the plugin level.

Recommendation

Update configId and profitDistributor atomically through a combined admin flow, or otherwise enforce a two-step migration procedure that prevents the plugin from operating while the pair is inconsistent. At minimum, the contract should avoid exposing a live state in which the new distributor is used with the old configId, or the old distributor is used with the new configId.

Client's Commentary:

Acknowledged

3. About MixBytes

MixBytes is a leading provider of smart contract auditing and blockchain security research, helping Web3 projects enhance their security and reliability.

Since its inception, MixBytes has been dedicated to safeguarding innovation in **DeFi** through rigorous audits and cutting-edge technical research.

Our team brings together experienced engineers, security auditors, and blockchain researchers with deep expertise in smart contract security and protocol design.

With years of proven experience in Web3, MixBytes combines in-depth technical excellence with a proactive, security-first approach.

Why MixBytes

- **Proven Track Record:** Trusted by leading blockchain protocols such as **Lido**, **Aave**, **Curve**, **1inch**, **Fluid**, **Gearbox**,**Resolv**, and others. MixBytes has successfully secured billions of dollars in digital assets.
- **Technical Expertise:** Our auditors and researchers hold advanced degrees in cryptography, cybersecurity, and distributed systems, backed by years of hands-on experience.
- **Innovative Research:** MixBytes actively contributes to blockchain security research and open-source tools, sharing insights with the global community.

Our Services

- **Smart Contract Audits:** Comprehensive security assessments of DeFi protocols to detect and mitigate vulnerabilities before deployment.
- **Blockchain Research:** In-depth technical studies, economic and protocol-level modeling for Web3 projects.
- **Custom Security Solutions:** Tailored frameworks and advisory for complex decentralized systems and blockchain ecosystems.

Contact Information

 https://mixbytes.io/

 https://github.com/mixbytes/audits_public

 https://x.com/MixBytes

 hello@mixbytes.io